
IMPROVING CLASSIFICATION OF OPEN ANIMAL TRACKS

TECHNICAL REPORT



Department of Computer Science
Aarhus University
Aarhus, Denmark



Department of Computer science
Aarhus University
Aarhus, Denmark

December 3, 2025

ABSTRACT

This paper investigates the challenge of classifying images of animal footprints, which is a relatively unexplored area. We make use of the ResNet-18 model and attempt to improve its performance through an iterative work process. The OpenAnimalTracks dataset is highly imbalanced, leading to persistent overfitting challenges. To handle these issues, we apply techniques such as regularization, dropout, and data augmentation. Grad-CAM visualization revealed that the model may rely more on image aspect ratios than on the actual footprint shapes, indicating potential biases in feature learning. Ultimately, our model outperforms a ResNet-50 model reported in related work.

Keywords Deep Learning · Visual Recognition · OpenAnimalTracks

1 Introduction

In the field of wildlife conservation, deep learning for visual recognition is increasingly valuable, as it offers non-invasive tools to understand animal behavior, presence, and population dynamics. It is especially important to look at animal habitat surveys, as they are essential for understanding and preserving biodiversity [1]. Many tools are used in this field to monitor wildlife, among which are animal footprints. They serve as a non-invasive tool for detecting an animal's presence and movement. That is why animal footprints must be accurately identified; however, doing so manually is hard work and requires great skill to distinguish among some of the different animal footprints.

In this paper, we explore the problem of classifying various animal footprints using neural networks. Although footprints are very useful for gaining knowledge about animals, automatic recognition has been limited due to a lack of public datasets. This lack of data makes it difficult to apply computer vision methods to footprint identification. The introduction of the OpenAnimalTracks [1] dataset helps to address this challenge. It is the first publicly available labeled dataset to facilitate the automated classification and detection of animal footprints. With the use of this dataset, our project aims to classify the footprints of 18 different animal species under varying conditions. Although the dataset contains a large number of images, it also presents several unavoidable challenges. Some footprints look very similar across species; the ground textures vary (mud, sand, and snow), and many footprints are unclear or incomplete.

A reason why this problem is interesting is that, from a deep learning perspective, it has not been widely studied. Animal footprint recognition is relatively unexplored compared to the well-known image classification tasks. This gives us the opportunity to test how well modern models generalize to a new and challenging type of data.

This introduces the research question for this project: How can a ResNet-based model be improved in terms of performance for the classification of animal footprints in the OpenAnimalTracks dataset, despite challenges such as species similarity, variable ground textures, and imbalance in the dataset?

2 Related Work

Prior research on automated wildlife monitoring has primarily focused on detecting or classifying animals in images or videos. Datasets such as these have made it possible to develop deep learning models for training in species identification, behavior analysis, and ecological monitoring [1]. However, these approaches rely on clear visual images, which can be difficult to obtain due to occlusion or animals being active at night.

Another way of studying and analyzing wildlife is through animal footprints. By observing only the footprints, it is possible to reveal the species and movements without needing to photograph the animals themselves, potentially avoiding intrusion into their natural habitat. Earlier research on footprint recognition has mostly relied on the manual annotation of footprints [1]. While this method may work well, it is very time-consuming, as experts must mark each footprint by hand.

A limitation of earlier animal footprint recognition is that the images are not publicly available, and one method used involves landmark coordinates, which means that deep learning models cannot easily be trained on raw footprint images. One study by Zoe Jewell and Sky Alibhai from 2013 [2] about identifying endangered species from footprints uses their so-called footprint identification technique (FIT). It involved experts manually placing landmarks at specific points on each image, which was the most accurate method at that time. They also added that they were working on a better method, as their aim is to produce a fast, cheap, reliable, and user-friendly tool to monitor species using footprints.

To address this gap, Shinoda and Shiohara [1] introduced OpenAnimalTracks, the first publicly available dataset designed for animal footprint recognition. It consists of 3579 animal footprint images across 18 species under various environmental and texture labels (mud, sand, and snow) (see Figure 1). The dataset has already been preprocessed and annotated.



Figure 1: Texture annotations. Divided texture background into mud, sand, and snow.

The authors also provided benchmarks for both species classification and footprint detection. The results show that attention-based architectures, in this case, the Swin Transformer, achieved the best classification performance with an average accuracy of 69.41%.

Our project builds upon the OpenAnimalTracks dataset to train deep learning models for identifying animal footprints. Previous research has aimed to reduce the dependency on expert-level annotations, particularly with the availability of animal footprints, which now enable automated methods that do not rely on manually placed landmarks. However, despite this progress, the existing benchmarks still leave room for improvement. In their benchmark, the best-performing classification model (Swin Transformer) reaches 69.41% accuracy across species, while the ResNet-50 model only achieved 62.84%, which is somewhat low and, as the authors note, leaves clear room for improvement in future

work. Our work contributes to the further exploration of modern deep learning techniques for accurate animal track classification.

3 Methods

3.1 The dataset

The Open Animal Tracks dataset consists of 3579 images across 18 different species. The dataset has both raw and cropped versions. The raw versions can usually consist of multiple footprints of the same or different animals, while the cropped versions crop the images to include only a single footprint with a small margin around it. The images used for the dataset differ across several conditions, such as lighting, substrate, resolution, rotation, and clarity. The biggest difference between the images is the substrate; some are on a very rocky substrate with a lot of texture, while others have a very smooth texture, like clay or snow.

The images vary in size, ranging from about 32 x 32 pixels to approximately 1000 x 1000 pixels. The images also do not have the same aspect ratio; while most are 1:1, some are closer to 2:1.

	Bear	Beaver	Cat	Coyote	Deer	Elephant	Fox	Goose	Horse	Lion	Mink	Mouse	Otter	Raccoon	Rat	Skunk	Squirrel	Turkey	Total
Classification																			
#Train	186	96	171	123	194	32	126	141	71	95	139	129	100	317	116	122	251	105	2514
#Val	27	12	28	17	28	3	15	16	12	13	17	24	11	37	21	14	32	19	346
#Test	53	26	53	32	50	10	39	36	14	26	50	38	30	79	36	40	76	31	719
Total	266	134	252	172	272	45	180	193	97	134	206	191	141	433	173	176	359	155	3579

Table 1: Comparison of class distributions for the Classification and Detection datasets.

The total number of images for each class is represented in Table I. Most classes have between 100 and 200 images, while the lowest has 33 images and the largest has 433 images. This makes the dataset very unbalanced. The class with the fewest number of images is the elephant class, which only has 3 images in the validation set, making it hard to determine the actual accuracy for that class.

We decided to follow the same split between training, validation, and testing as was done in the OpenAnimalTracks paper [1], with a ratio of 7:1:2.

3.1.1 Data augmentation

Given that the total dataset is on the smaller side and that some classes have a very limited number of images. We would likely face problems with overfitting. Our first plan to prevent overfitting was to add data augmentation. The data augmentation would modify the existing images, resulting in an increased number of images for us to train the model.

To augment the images, we used a variety of transforms. This could vary from rotating and flipping, to changing the color. Throughout our training, we found what seemed to work best for us: changing the images to grayscale, which reduced the influence of varying backgrounds, rotating the images by a random value, possibly flipping them either horizontally or vertically, randomly applying blur or adjusting the sharpness, and lastly, randomly changing the brightness, contrast, saturation, and hue.

All of these were done to mimic what an animal track could look like under different rotations, different lighting condition, and different colored substrates.

Seeing as the pictures do not have the same aspect ratio and their sizes vary significantly, we also have to make sure they are all the same size and aspect ratio. We therefore resized all images so that their largest side was 128 pixels and then padded the images to be 128 x 128.

3.2 Network Architecture

For our deep learning model, we decided to use ResNet-18 because it is lightweight, trains quickly, and is less likely to overfit compared to larger ResNet variants, which is an advantage for our relatively small dataset. Choosing ResNet-18 over other models was more of a random choice. Although simpler, it is faster for training and still captures the necessary features in our footprint images.

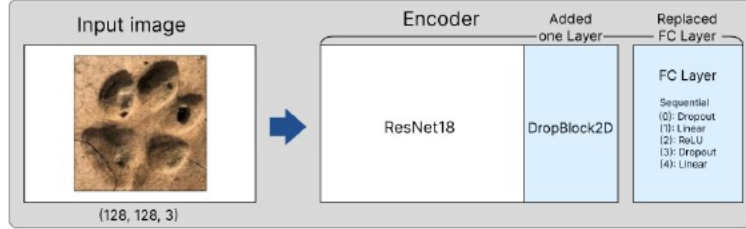


Figure 2: Network architecture

We customized the final part of the network for our task. Instead of using the default fully connected layer, we replaced it with a sequential module tailored to our 18 animal classes. It includes two Dropout layers for regularization, a Linear layer (256 output features) followed by a ReLU activation, and another Linear layer mapping to the 18 output classes.

We also added an additional DropBlock2D layer and integrated it into the conv2 layers of the BasicBlock modules within layer4. This regularization technique randomly drops contiguous spatial blocks within feature maps. This way, the model learns more distributed features and prevents it from relying on specific local patterns in the image.

3.3 Loss Function

For this project, we used Cross-Entropy loss to train the model. This loss function fits our task because we classify images into 18 different animal-track categories, where only one class is correct for each image. During training, the loss was computed for every batch and was used to update the model's weights. The development of the training and validation loss over time guided several decisions in the project, including learning-rate tuning and when to stop training. A consistent decrease in loss indicated that the model was improving its ability to distinguish between the different animal tracks.

Because the dataset was imbalanced, we also experimented with label smoothing. We applied label smoothing to make the model less sensitive to overconfident predictions and to improve generalization. Instead of training the network with a vector in which the correct class has the value 1.0 and the rest have the value 0.0, label smoothing decreases the correct value to, e.g., 0.9 and then distributes the remaining probability to the other classes. In practice, this helped our model become less prone to overfitting due to the model not becoming overly confident, especially in cases where certain animal tracks might look very similar.

3.4 Transfer Learning

Transfer learning was used to train our model. Transfer learning allows us to use already initialized weights from previously trained models. We used the ResNet-18 weights trained on the ImageNet1K dataset [3]. This makes training the model faster, as we do not have to train all layers from scratch. Transfer learning works as a feature extractor since the pretrained model is well adapted to detect different edges, textures, and shapes.

Due to our smaller dataset, transfer learning was especially useful for learning meaningful features more quickly with less data. This leads to faster convergence and better overall performance compared to training a network from scratch.

3.5 Optimizer

The adaptive optimization algorithm (Adam) with momentum is used as our optimizer. Adam computes an adaptive learning rate for each parameter, whereas Stochastic gradient descent (SGD), which we also tried, maintains only a single learning rate that does not change during the course of training. Although SGD generalizes better, our decision to choose the Adam optimizer was based on its superior performance during testing.

Adam uses the loss computed after each batch to update the model parameters. We used the optimizer in conjunction with learning-rate scheduling. Learning rate scheduling allows us to change the overall learning rate during training to ensure that training progresses smoothly. Furthermore, it can help us escape local minima and reach even further into the loss landscape.

For our learning-rate scheduling, we tried several different types of schedulers. We tried to reduce the learning rate on plateau, cyclic learning rate, exponential learning rate, linear, and sequential schedulers. After testing, our final approach settled on using a sequential scheduler that included a warm-up scheduler and a primary scheduler. The warm-up scheduler was a linear scheduler that, over the course of approximately 5 epochs, would transition from a baseline learning-rate to the maximum learning-rate. The warm-up scheduler is used to stabilize early training, as a high learning rate can diverge or cause damage to useful pretrained features. After the warm-up scheduler, we switch to an exponential scheduler that decreases the starting learning rate by gamma with each batch.

3.6 Dropout

As overfitting still seemed to be a problem, we further attempted to decrease it by implementing dropout. Both neuron level dropout and channel level dropout. The neuron level dropout was added in the final fully connected layer, as adding it to the convolutional layers is ineffective [4]. To further increase the generalization of the model, we added a dropout layer just before the final fully connected layer and after the fully connected layer. This helped regularize the input features coming to the fully connected layer and the hidden representations after the first linear layer. Additionally, to prevent the model from becoming overly reliant on certain larger features of the images, we added a channel level dropout.

3.7 Work process

To select our final methods, we used an iterative workflow. Throughout the development of our model, we identified several issues, prioritized the most critical ones, such as overfitting, analyzed the potential causes of this, and tested some solutions. The same iterative process guided us when we encountered slow training speeds, which led us to reduce the image sizes to accelerate experimentation. This cycle of diagnosing and prioritizing our problems, designing solutions, and testing them helped us achieve the best results quickly.

4 Results

Our model, ResNet-18, was trained for 35 epochs using the sequential learning rate scheduler, which employs a linear warm-up followed by exponential decay. The results are shown in Table 2 along with a comparison to existing results from the OpenAnimalTracks paper [1]. Here, it is important to note that the best results from the paper were obtained using the ResNet-50 and Swin-B models compared to our model, which uses ResNet-18. However, our ResNet-18 model outperformed their ResNet-50 model while falling short of the Swin-B model.

Additionally, the model reached a training accuracy of 88% with a training loss of 0.8021 at the end of training. The highest validation accuracy we achieved was 73.12%, but the final test accuracy after the final epoch was 65.79% with a validation loss of 1.345.

	Bear	Beaver	Cat	Coyote	Deer	Elephant	Fox	Goose	Horse	Lion	Mink	Mouse	Otter	Raccoon	Rat	Skunk	Squirrel	Turkey	Mean
Our Results:																			
ResNet-18	69.811	57.692	64.151	68.75	86.0	50.0	61.538	75.0	78.571	69.231	66.0	55.263	63.333	73.418	52.778	62.5	48.684	80.645	65.743
Existing work:																			
ResNet-50	66.04	50.00	52.83	46.88	77.78	71.43	74.36	73.08	68.00	52.63	78.00	50.00	70.00	49.37	61.11	55.00	53.95	80.65	62.84
Swin-B	60.38	57.69	60.38	78.12	80.56	64.29	79.49	76.92	72.00	73.68	86.00	50.00	76.67	58.23	69.44	65.00	56.58	83.87	69.41

Table 2: Our results vs. Existing work results for All Species.

In Table 3 we have calculated the precision, recall, and F1-score, showing only the results. Here, our model achieved an overall test accuracy of 66%, with a macro F1-score of 0.68, indicating moderate performance across all species.

Class	Precision	Recall	F1-score	Support
Species	...	...	...	...
accuracy		0.66		719
macro avg	0.72	0.66	0.68	719
weighted avg	0.68	0.66	0.66	719

Table 3: Classification report for all species.

Hyperparameter	Value
learning-rate	3e-4
batch_size	36
dropout	0.15
dropout2	0.15
drop_block_prob	0.15
label_smoothing	0.05
gamma	0.999
sequential_milestone	400

Table 4: Training hyperparameters.

The optimal results were achieved by adjusting the hyperparameters, as seen in Table 4. We used a maximum learning rate of 3×10^{-4} with warmup before and exponential decay after, as well as a batch size of 36. The dropout layers have values of 0.15 and 0.15, with a dropblock probability of 0.15 to apply regularization. Label smoothing (0.05) further improved generalization. To ensure gradual convergence, the learning-rate schedule is 0.999, and the sequential milestone is 400. Together, these hyperparameters were tuned repeatedly until they achieved the highest validation accuracy.

4.1 Evaluation

To investigate how well our model is at predicting, we visualized the confusion matrix of ResNet-18 in Figure 3. It generally has a good diagonal line, but some misclassifications have occurred. Among these misclassifications, the most notable are otter, beaver, and mouse being mistaken for raccoon, as well as skunk being mistaken for western gray squirrel. There are also some low diagonal values, such as mouse (0.45), elephant (0.50), rat (0.50), and coyote (0.53), indicating that the model struggles with these classes due to limited training examples, similarity to other animals, or extremely small and faint footprints. To compare this to the OpenAnimalTracks paper [1], the authors observed misclassifications, with coyotes being mistaken for foxes and mouse footprints tending to be misclassified as minks.

Another important aspect to consider when evaluating the model’s performance is looking at the model accuracy and loss in Figure 4. The loss should decrease over time, which it does up to a certain epoch; however, it stops improving significantly thereafter. The validation accuracy is around 70%, which is slightly lower than the training accuracy of around 90%.

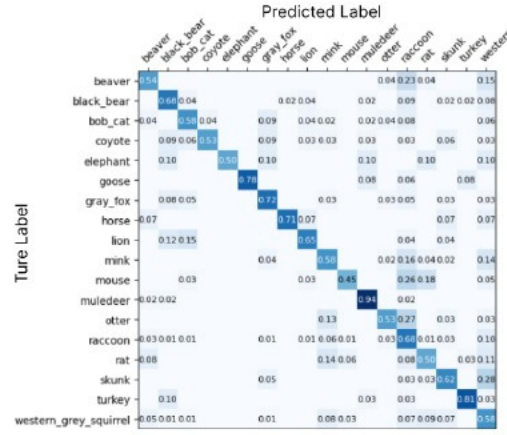


Figure 3: Confusion Matrix (empty squares represent the value 0.0)

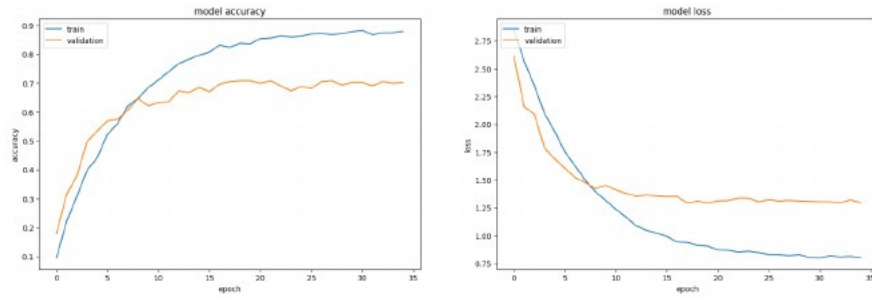


Figure 4: Model Accuracy and Loss

5 Discussion

The final model did not fully meet our expected results. We achieved good results that we were very excited about, such as achieving a higher score than they did in OpenAnimalTracks [1] using a similar model, the ResNet-50; however, compared to their Swin-B model, which also utilizes the same dataset, we achieved worse results. There are several reasons why we did not meet our expectations, but several key factors likely contributed. Firstly, the dataset is slightly small in size, very imbalanced, and has high variation in the quality of the footprint, with many classes that need to be handled well to avoid bias. Secondly, the model we used may not have been the best fit for our dataset.

We compare our model to the highest achieving model in the OpenAnimalTracks paper [1] on the same dataset, which outperforms ours and indicates that we may not have used the best model.

5.1 OpenAnimalTracks dataset

As noted in the methods section [3] our dataset exhibits considerable class imbalance, which may limit model accuracy despite our efforts to counteract it. One strategy we did not consider was reducing the number of classes. Removing outlier categories, such as the elephant class, might have reduced noise and helped the model form more reliable predictions.

When inspecting the images closely from the dataset, they exhibit varying quality. Some images contain multiple footprints on top of each other, while others contain footprints from another species within the same footprint. Furthermore, the images are cropped based on the shape of the footprint, so if the footprint is longer than it is wide, the input image will be as well. This can result in the model learning the aspect ratio of the image instead of learning about the actual footprints.

5.2 Test Results and Model comparison

In the Classification report, shown in Table 3, our model correctly classifies 66% of all 719 test samples across all species. The macro average has a precision of 72% and a recall of 66%, indicating that the model is more accurate when it predicts a class but misses more true samples. The F1-score drops closer to recall because it penalizes imbalances. The macro average also has better results than the weighted average, indicating that the model performs better when it treats each species equally, regardless of how many samples each class has. From these results, we can compare the model validation accuracy with its training accuracy, as well as examine the corresponding losses in Figure 4. In this figure, the model seems to be overfitting slightly after around 10 epochs. The model does not improve much on unseen data (validation accuracy) but continues to get better on the training dataset. Also, the loss of the validation set is higher than that of the training set after the 10 epochs, which means that the model becomes better at predicting the training data rather than unseen data. Additionally, when adjusting our hyperparameters to prevent overfitting, we achieve worse validation accuracy. Although the model seems to overfit, the results, compared to our baseline model of around 40% test accuracy, show drastic improvements.

To understand the performance gap from the results section 4 in Table 2 we compare our ResNet-18 model against the Swin-B model used in the OpenAnimalTracks paper 11. There is clearly room for improvement in our model, as the Swin-B model outperforms the ResNet-18 model. One reason the Swin-B model might be performing better than ResNet-18 is due to its feature extraction capabilities. ResNet-18 is a traditional Convolution Neural Network (CNN) that, despite its residual block design for mitigating vanishing gradients, relies on local features, while Swin-B is a Vision Transformer that uses a shifted window mechanism to capture both local features and global context 5. Swin-B is also an attention-based model, while ResNet is a convolution-based model. In the paper OpenAnimalTracks 11, the authors discuss that Swin-B outperforms ResNet because attention-based models tend to focus on the structures of images rather than on textures. The use of attention-based models, such as Swin-B and ViT-B, might have resulted in better outcomes for us.

5.3 Grad-CAM

Grad-CAM visualizations were generated for the six best and six worst predictions (based on loss) in the test dataset to examine where the model directs its attention. Our best results were achieved using the 'With Padding' approach, as shown in Table 5. However, when looking at the attention maps in the 'With Padding' column, we would expect the footprint to be marked; instead, the area around the footprint is given the most weight, especially in the corners. The most likely cause of this could be the non-square input images, which were not cropped; as a result, the model implicitly learned the aspect ratio in connection with the overall footprint outline as part of the classification features.

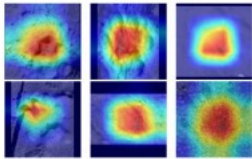
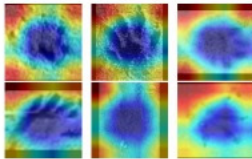
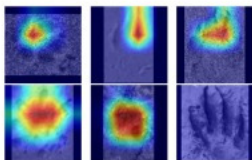
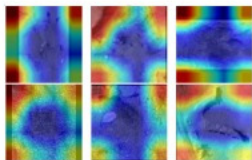
	RandomCrop	With Padding
6 best predictions		
6 worst predictions		

Table 5: Two different approaches to transforming the data. RandomCrop does a crop of the image in a random size(output can be zoomed in) With padding used the entire image without zooming or cropping the image

References

- [1] R. Shinoda and K. Shiohara, “Openanimaltracks: A dataset for animal track recognition,” in *2024 IEEE International Conference on Image Processing (ICIP)*. IEEE, 2024, pp. 110–116.
- [2] Z. Jewell and S. Alibhai, “Identifying endangered species from footprints,” *International Society for Optics and Photonics (SPIE) Newsroom*, vol. 2013, pp. 1–3, 2013.
- [3] PyTorch team, *torchvision.models.resnet18 — Torchvision Documentation*, PyTorch, 2025, uRL: <https://docs.pytorch.org/vision/main/models/generated/torchvision.models.resnet18.html> (accessed: 2025-11-27).
- [4] S. Cai, Y. Shu, G. Chen, B. C. Ooi, W. Wang, and M. Zhang, “Effective and efficient dropout for deep convolutional neural networks,” 2020. [Online]. Available: <https://arxiv.org/abs/1904.03392>
- [5] H.-R. Chang, S.-W. Chen, and G.-J. Weng, “Deep learning for biodiversity monitoring: A comparative study on wildlife image classification,” in *2025 IEEE International Conference on Consumer Electronics - Taiwan (ICCE-Taiwan)*, 2025, pp. 571–572.